WG01: Kernel C++: A Methodology for Freestanding Development

Amlal El Mahrouss

December 2025

Abstract

Many Operating Systems Kernels have been shipped using the C programming language.
And some of them like EKA2 use the C++ programming language. Although notoriously difficult, one may still adapt to those constraints in order to deliver one such operating system kernel. That is the reason that most production-grade kernels (Linux, XNU, and NT) are mostly written in C. With a higher-level subset in C++. However, when correctly applying C++ principles to kernel development, one makes the development much more easier to pull off.

1 The Three Principles of Kernel C++.

1.1 Part One: The C++ Runtime.

The problem mostly lies in the C++ runtime. Which assumes an existing host. A host is the contrary of a freestanding target, that is a program which expects a runtime to be present and linked to the program. A C++ Kernel may instead make use of compile-time features of C++ alongside a tiny C++ runtime to make sure that no issues arise because of this host/freestanding difference.

1.2 Part Two: Constexpr and Friends.

One may avoid V-tables or runtime when possible. While focusing instead on meta-programming and compile-time features offered by C++. For example one may use templates to implement a scheduling policy algorithm. One example of such implementation may be:

```
/// Reference Implementation: https://github.com
/// /nekernel-org/nekernel/blob/stable/src/kernel/
/// KernelKit/CoreProcessScheduler.h#L51-L76
/// AND: https://github.com/nekernel-org/nekernel/blob/stable/src/kernel
/// /KernelKit/CoreProcessScheduler.h#L78-L105

struct FileTree final {
    static constexpr bool is_virtual_memory = false;
    static constexpr bool is_memory = false;
    static constexpr bool is_file = true;
    /// ...
};

struct MemoryTree final {
    static constexpr bool is_virtual_memory = false;
    static constexpr bool is_memory = true;
    static constexpr bool is_file = false;
    /// ...
};
```

As you see, two structures leverages the ‘constexpr’ keyword to make sure no bugs or panic occur at runtime because of a misuse of a system resource.

Which is why the constexpr keyword is very powerful here, we avoid the many pitfalls of writing (and hoping) that the C version will be well-thought enough so that we can catch such bugs later.

1.3 Bonus: Using Concepts and the DDK.

This bonus subsection intends to show how properly applied C++
Can solve issues related to driver validation. The following example shows how powerful C++ can be when combining it with Device Driver development as well.

```
/// Reference implementation:
/// https://github.com/nekernel-org
/// /nekernel/blob/stable/src/libDDK/DriverKit
/// /c%2B%2B/driver_base.h
/// @brief This concept requires the driver
/// to be IDriverBase compliant.

inline constexpr auto kInvalidType = 0;

template <class Driver>
concept IsValidDriver = requires(Driver drv) {
  { drv.IsActive() && drv.Type() > kInvalidType };
};

/// @brief Consteval helper to detect
/// whether a template is truly based on IDriverBase.
/// @note This helper is consteval only.
template<IsValidDriver T>
consteval void ce_ddk_is_valid(T) {}
```

1.4 Part Three: Memory and C++ Classes.

This last part treats about the final and most important part of this paper so far. Memory. As you may already have known, the C++ language uses a class lookup system (also called v-table) in order to refer to a base method in case if the instance has it missing.

```
/// Link: https://godbolt.org/z/aK6Y98xnd
/// /std:c++20 /Wall

#include <iostream>

class A
{
public:
    explicit A() = default;
    virtual ~A() = default;

    virtual void doImpl()
    {
        std::cout << "doImpl()\r\n";
    }
};

class B : public A
{
public:
    explicit B() = default;
    ~B() override = default;
};

int main() {
    B callImpl;
    callImpl.doImpl();
```

```
}
```

2 Addressing V-Tables in a C++ Kernel.

Now the problem with kernel development is that we want to avoid such feature as much as possible, and we'd do that by following the Prong on Inheritance:

2.1 The Three Prongs on Inheritance Decision Framework.

TTPI is a thought exercise used to decide whether you should consider using C++ in a kernel. Consider the following thought exercise:

1: Is this a feature that can be implemented with other similar protocols/concepts?

2: Is this doable without too much trade-off costs?

3: Is this doable without V-Tables?

If 2/3 of those questions fail, you should consider finding another solution to your problem. As it surely has an equivalent without the problematic aspects.

2.2 The Vettable Pattern, validating containers for Freestanding C++.

The following concept makes sure that the 'class T' is truly vettable by the Operating System Kernel. Such properties as called 'Vettable' in NeKernel.

```
template <class T, typename OnFallback>
concept IsVettable = requires(OnFallback fallback) {
  { Vettable<T>::kValue ? true : fallback() };
};
```

The vettable structure makes use of template metaprogramming in modern C++ in order to evaluate whether a container is vettable or not, such containers inherits the 'IVettable' structure and are marked final.

```
struct IVettable {
  explicit IVettable() = default;
  virtual ~IVettable() = default;

  NE_COPY_DEFAULT(IVettable)
};
```

A vettable system may look as such in a freestanding C++ system.

```
#define NE_VETTABLE final : public ::Kernel::IVettable
#define NE_NOT_VETTABLE final : public ::Kernel::INotVettable

namespace Kernel {
/// @brief Vet interface for objects.
struct IVettable {
  explicit IVettable() = default;
  virtual ~IVettable() = default;

  NE_COPY_DEFAULT(IVettable)
};

struct INotVettable {
  explicit INotVettable() = default;
  virtual ~INotVettable() = default;

  NE_COPY_DEFAULT(INotVettable)
};

template <class Type>
struct Vettable final {
```

```
  static constexpr bool kValue = false;
};

template <>
struct Vettable<INotVettable> final {
  static constexpr bool kValue = false;
};

template <>
struct Vettable<IVettable> final {
  static constexpr bool kValue = true;
};

/// @brief Concept version of Vettable.
template <typename T, typename OnFallback>
concept IsVettable = requires(OnFallback fallback) {
  { Vettable<T>::kValue ? true : fallback() };
};

template <class Type, typename OnFallback>
concept IsNotVettable = requires(OnFallback fallback) {
  { !Vettable<Type>::kValue ? true : fallback() };
};
}  // namespace Kernel
```

Source https://github.com/nekernel-org/nekernel/blob/stable/src/kernel/NeKit/Vettable.h

3 Conclusion

We can now conclude that C++ in Kernel Development is indeed possible under a subset of C++, here called Kernel C++. A reference implementation of this paper exists, It's called NeKernel.org, available under the same internet address. We are looking forward to any questions or inquiries at: contact@nekernel.org.

4 References

1. NeKernel.org (2025). NeKernel Operating System. Available at: `https://nekernel.org`
2. Sales, J., Tasker, M. (2005). Introducing EKA2. Symbian OS Internals. Wiley. Available at: `https://media.wiley.com/product_data/excerpt/47/04700252/0470025247.pdf`
3. Driesen, K., Hölzle, U. (1996). The direct cost of virtual function calls in C++. *OOPSLA '96*. ACM. DOI: 10.1145/236338.236369